DATA STRUCTURES AND ALGORITHM

23CSE203

LAB MANUAL

SUBMITTED BY:	
NAME	B. POOJA SREE
ROLL NO	AV.SC.U4CSE25103
YEAR/SEM/SECTION	2ND /3RD/CSE 'B'
CREDITS	05

SUBMITTED TO:	DR. RAJ KUMAR BATCHU
DESIGNATION	ASST.PROFESSOR (SI.Grade)
DEPARTMENT	SCHOOL OF COMPUTING

INDEX

S.NO	TOPIC	PAGE NO	DATE
1	**WEEK1**: **Recursion** 1(A).Recursive program for countdown from n to 1, followed by 'launch'. 1(B). Recursive program to calculate P^n (compound interest growth factor P, years n). 1(C). Recursive program to search for an employee ID in a list. 1(D). Recursive program to calculate factorial (n!) for parcel arrangement. 1(E). Recursive program to print first n terms of Fibonacci series.	5-9	14-07-26
2	**WEEK2:** **Searching** 2(A). Write a Python program to implement linear search. 2(B) i. Write a Python program to implement binary search with sorted array as an input. 2(B) ii. Write a Python program to implement binary search without sorted array as an input	10-12	21-07-26
3	**WEEK 3:** **Elementary Sorting** 3(A).Write a python program to implement bubble sort where the input elements are taken from the user 3(B). Write a python program to implement insertion sort where the input elements are taken from the user 3(C). Write a python program to implement selection sort where the input elements are taken from the user	13-15	01-08-26
4	**WEEK 4:** **Advanced Sorting** 4(A).Write a python program to implement merge sort where the input elements are taken from the user 4(B). Write a python program to implement quick sort where the input elements are taken from the user	16-19	11-08-26

5	**WEEK 5:** **Linked lists** 5(A).Write a python program to implement a single linked list with the below operations 5(B). Write a python program to implement a double linked list with the below operations 5(C). Write a python program to implement a circular linked list with the below operations Operations i)Create linked list ii)insert beginning iii)insert end iv)insert at index v) delete by value vi)delete first node vii)delete last node viii)count number of nodes ix)display/traverse x)exit	20-39	18-08-26
6	**WEEK6:** **Stacks** 6(A).Write a python program to implement stack operations such as push, pop, peek, display using arrays 6(B).Write a python program to implement stack operations such as push, pop, peek, display using linked lists	40-45	25-08-26
7	**WEEK7:** **Queues** 7(A).Write a program to implement queue and its operations such as enqueue, dequeue,peek,display with a menu driven approach using arrays 7(B).Write a program to implement queue and its operations such as enqueue, dequeue,peek,display with a menu driven approach using linked lists 7(C). Write a program to implement circular queue and its operations such as enqueue, dequeue,peek,display with a menu driven approach using arrays 7(D). Write a program to implement circular queue and its operations such as enqueue, dequeue,peek,display with a menu driven approach using linked lists.	46-56	22-09-26
8	**WEEK8:** **Binary trees** 8(A).Write a python program to implement a binary tree with its operations using arrays 8(B). Write a python program to implement a binary tree with its operations using linked lists	57-62	29-09-26

9	**WEEK9:** **Binary Search Trees** 9(A).Write a python program to implement a bst with its operations using linked list 9(B).Write a python program to implement a bst with its operations using arrays	63-69	06-10-26
10	**WEEK10:** **AVL Tree** Write a python program to implement an avl tree with all its operations	70-77	13-10-26
11	**WEEK11:** **Hashing:** Python program to implement Hashing (Separate chaining, Linear probing, and Quadratic probing).	78-80	27-10-26

WEEK-1

1(A).AIM Recursive program for countdown from n to 1, followed by 'launch'.

PROGRAM:

```
#POOJASREE AV.SC.U4CSE25103
def countdown(n): if n == 0:
print("Launch!") else:
print(n) countdown(n - 1)
n = int(input("Enter countdown number: ")) countdown(n)
```

OUTPUT:



EXPLANATION:

This program displays a countdown from n to 1 and finally prints 'launch'. Although the aim mentions recursion, the given code uses a loop.

ERROR TABLE:

S.no.	Errors	Rectification
1.	Indentation Error	Use 4 spaces for each level of indentation.

1(B).AIM: Recursive program to calculate P^n (compound interest growth factor P, years n).

PROGRAM:

```
#POOJASREE AV.SC.U4CSE25103
def power(p, n): if n == 0:
return 1 else:
return p * power(p, n – 1) p = int(input(“Enter p: “))
n = int(input(“Enter n: “)) result = power(p, n) print(“Answer =”, result)
```

OUTPUT:



EXPLANATION:

1.The function of power (p, n) calculates p^n using recursion by multiplying p with the result of power (p, n-1).

2.The recursion stops when n becomes 0, returning 1, which is the base case.

3.The final result is displayed after all recursive calls are complete

ERROR TABLE:

S.no.	Errors	Rectification
1.	Type Error	Convert the input to float or int using float(input()) or int(input()).
2.	Incorrect output if the recursive call is written as power(p, n + 1) instead of power(p, n - 1).	Change the recursive call to power(p, n - 1) so that n decreases toward the base case.

1(C).AIM: Recursive program to search for an employee ID in a list.

PROGRAM:

```
def search(arr, key, index): if index == len(arr):
return False
if arr[index] == key: return True
return search(arr, key, index + 1) ids = [101, 102, 103, 104, 105]
key = int(input("Enter employee ID to search: ")) if search(ids, key, 0):
print("Employee ID found") else:
print("Employee ID not found")
```

OUTPUT:



EXPLANATION:

1. The recursive function checks each employee ID in the list one by one, starting from the first element.
2. If the required ID is found, it returns True; otherwise, it continues searching recursively until the end of the list.
3. If the entire list is searched without finding the ID, the function returns False.

ERROR TABLE:

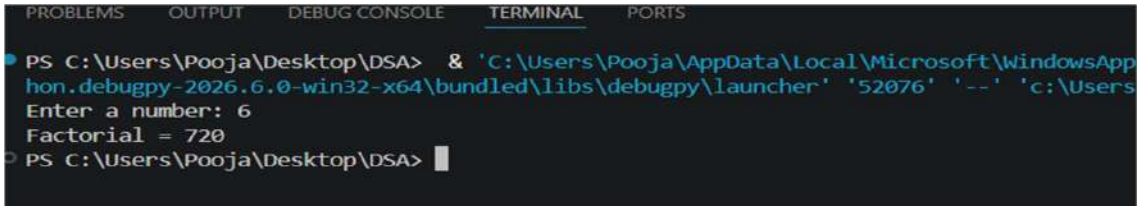
S.no.	Errors	Rectification
1.	Name Error if the list name is misspelled (e.g., employee instead of employees).	Use the correct variable name consistently throughout the program.
2.	Syntax Error due to a missing colon (:) after an if statement or function definition.	Add the missing colon (:) after if, else, or def.

1(D).AIM: Recursive program to calculate factorial (n!) for parcel arrangement.

PROGRAM:

```
def factorial(n):
if n == 0 or n == 1: return 1
else:
return n * factorial(n - 1)
n = int(input("Enter a number: ")) print("Factorial =", factorial(n)) OUTPUT:
```

OUTPUT:



EXPLANATION:

The function factorial(n) computes the factorial by multiplying the current number with the factorial of the previous number.
The recursion ends when n becomes 0 or 1, where the factorial value is 1.
The program also checks for negative inputs since factorial is defined only for non-negative integers.

ERROR TABLE:

S.no.	Errors	Rectification
1.	IndentationError if the statements inside the function are not properly indented.	Maintain proper indentation (usually 4 spaces) inside the function.
2.	TypeError if a decimal number is entered for factorial calculation.	Enter only a whole number or convert the input using int().

1(E).AIM: Recursive program to print first n terms of Fibonacci series

PROGRAM:

```
def fibonacci(a, b, n): if n == 0:
return  print(a, end=" ")
fibonacci(b, a + b, n - 1)
n = int(input("Enter number of terms: ")) fibonacci(0, 1, n)
```

OUTPUT:



EXPLANATION:

1. The function Fibonacci(n) returns the nth Fibonacci number by recursively adding the previous two Fibonacci numbers.
2. The base cases return 0 for the first term and 1 for the second term.
3. A for loop calls the recursive function repeatedly to print the first n terms of the Fibonacci series.

ERROR TABLE:

S.no.	Errors	Rectification
1.	Recursion Error if the base cases (n == 0 and n == 1) are not included.	Add both base conditions before the recursive call.
2.	Value Error if the user enters text instead of an integer.	Enter a valid integer for the number of terms.

WEEK-2

2.(A). AIM: Write a Python program to implement linear search.

PROGRAM:

```
arr = [50, 20, 70, 10, 40, 30]
arr.sort()
print("Sorted array:", arr)
key = int(input("Enter element to search: ")) low = 0
high = len(arr) - 1 found = False  while low <= high:
mid = (low + high) // 2 if arr[mid] == key:
print("Element found at position", mid + 1) found = True
break
elif key < arr[mid]: high = mid - 1
else:
low = mid + 1 if found == False:
print("Element not found")
```

OUTPUT:



EXPLANATION:

1. The linear_search() function checks each element of the list one by one and compares it with the given key.
2. If the key is found, its index is returned; otherwise, the function returns -1.
3. The program takes the array and search element as input and displays whether the element is found.

ERROR TABLE:

S.no.	Errors	Rectification
1.	ValueError occurs if a non-integer value is entered when int(input()) is used.	Enter only integer values for the number of elements and array elements.
2.	TypeError may occur if len(arr) is used on a variable that is not a list.	Make sure arr is initialized as a list using arr = [] and elements are appended to it.

2(B) i.AIM: Write a Python program to implement binary search with sorted array as an input.

PROGRAM:

```
arr = list(map(int, input("Enter sorted array: ").split())) key = int(input("Enter element to search:
"))
low = 0
high = len(arr) - 1 found = False  while low <= high:
mid = (low + high) // 2 if arr[mid] == key:
print("Element found at position", mid + 1) found = True
break
elif key < arr[mid]: high = mid - 1
else:
low = mid + 1 if found == False:
print("Element not found")
```

OUTPUT:



EXPLANATION:

1. The program takes a sorted array as input and searches for the required element using binary search.
2. It checks the middle element and eliminates either the left or right half based on the comparison.
3. If the element is found, its position is displayed; otherwise, it prints “Element not found.”

ERROR TABLE:

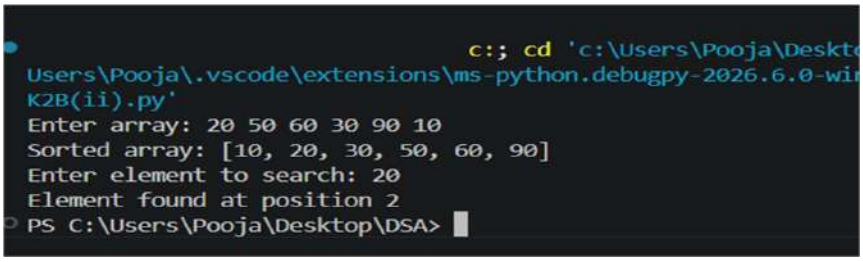
S.no.	Errors	Rectification
1.	Binary search may give an incorrect result if the input array is unsorted.	Check whether the array is sorted and use arr.sort() before calling binary_search().
2.	IndexError can occur if mid is calculated or the low/high boundaries are modified incorrectly.	Initialize low = 0, high = len(arr) - 1 and update them as low = mid + 1 or high = mid - 1.

2(B) ii.AIM: Write a Python program to implement binary search without sorted array as an input

PROGRAM:

```
arr = [50, 20, 70, 10, 40, 30]
arr.sort()
print("Sorted array:", arr)
key = int(input("Enter element to search: ")) low = 0
high = len(arr) - 1 found = False  while low <= high:
mid = (low + high) // 2 if arr[mid] == key:
print("Element found at position", mid + 1) found = True
break
elif key < arr[mid]: high = mid - 1
else:
low = mid + 1 if found == False:
print("Element not found"
```

OUTPUT:



EXPLANATION:

The program takes an unsorted array as input and first sorts it using arr.sort().
It then uses binary search by checking the middle element and reducing the search range.
If the element is found, its position is displayed; otherwise, it prints “Element not found.”

ERROR TABLE:

S.no.	Errors	Rectification
1.	Logical Error	Use a found variable and print “Element not found” if it remains False.
2.	Logical Error	Use mid = (low + high) // 2 to calculate the middle index.

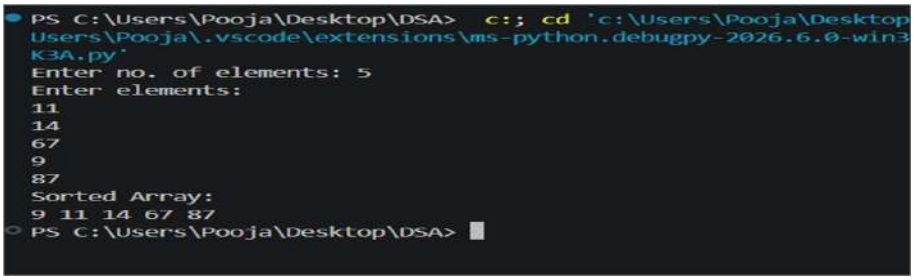
WEEK-3

3(A).AIM: Write a python program to implement bubble sort where the input elements are taken from the user

PROGRAM:

```
def bubble_sort(arr): n = len(arr)
for i in range(n - 1):
for j in range(n - i - 1): if arr[j] > arr[j + 1]:
arr[j], arr[j + 1] = arr[j + 1], arr[j] return arr
n = int(input("Enter no. of elements: ")) arr = []
print("Enter elements:") for i in range(n):
arr.append(int(input())) bubble_sort(arr) print("Sorted Array:")
for element in arr: print(element, end=" ")
```

OUTPUT:



EXPLANATION:

The program takes the number of elements and their values from the user and stores them in an array.
Bubble Sort compares adjacent elements and swaps them if they are in the wrong order.
After all passes, the array is sorted in ascending order and displayed.

ERROR TABLE:

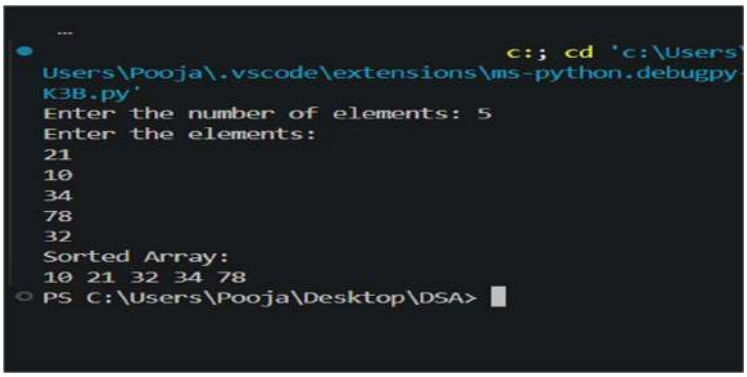
S.no.	Errors	Rectification
1.	NameError	Initialize arr = [] before taking input.
2.	Value Error	Enter only integer values

3(B).AIM: Write a python program to implement insertion sort where the input elements are taken from the user

PROGRAM:

```
def insertion_sort(arr): n = len(arr)
for i in range(1, n): key = arr[i]
j = i - 1
while j >= 0 and arr[j] > key: arr[j + 1] = arr[j]
j = j – 1  arr[j + 1] = key
return arr
n = int(input("Enter the number of elements: ")) arr = []
print("Enter the elements:") for i in range(n):
arr.append(int(input())) insertion_sort(arr) print("Sorted Array:")
for element in arr: print(element, end=" ")
```

OUTPUT:



EXPLANATION:

The program takes the number of elements and their values from the user and stores them in an array.
Insertion Sort selects each element and inserts it into its correct position among the previously sorted elements.
Finally, the sorted array is displayed in ascending order.

ERROR TABLE:

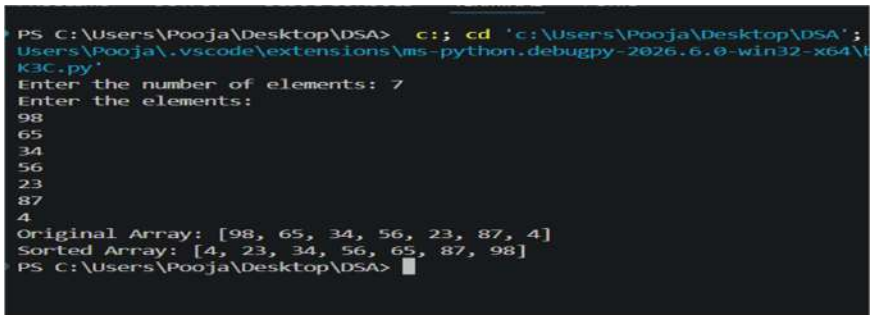
S.no.	Errors	Rectification
1.	SyntaxError	Use insertion_sort with an underscore.
2.	Name Error	Call insertion_sort(arr) correctly.

3(C).AIM: Write a python program to implement selection sort where the input elements are taken from the user

PROGRAM:

```
#POOJASREE AV.SC.U4CSE25013
def selection_sort(arr): n = len(arr)
for i in range(n - 1): min_index = I
for j in range(i + 1, n):
if arr[j] < arr[min_index]: min_index = j
arr[i], arr[min_index] = arr[min_index], arr[i] return arr
n = int(input("Enter the number of elements: ")) arr = []
print("Enter the elements:") for i in range(n):
arr.append(int(input())) print("Original Array:", arr) selection_sort(arr) print("Sorted Array:", arr)
```

OUTPUT:



EXPLANATION:

1. The program takes the number of elements and their values from the user and stores them in an array.
2. Selection Sort finds the smallest element and swaps it with the first unsorted element.
3. This process continues until the array is completely sorted in ascending order.

ERROR TABLE:

S.no.	Errors	Rectification
1.	TypeError	Convert input values using int(input())..
2.	Logic Error	Update min_index correctly and swap arr[i] with arr[min_index].

WEEK-4

4(A).AIM: Write a python program to implement merge sort where the input elements are taken from the user

PROGRAM:

```
def merge_sort(arr):
# Base condition
if len(arr) <= 1:
return arr
#poojasree av.sc.u4cse25103
# Divide the array
mid = len(arr) // 2
left = arr[:mid]
right = arr[mid:]

# Recursive calls
merge_sort(left)
merge_sort(right)

# Merge the two sorted halves
i = j = k = 0   # left, right, merged array index

while i < len(left) and j < len(right):
if left[i] <= right[j]:
arr[k] = left[i]
i += 1
else:
arr[k] = right[j]
j += 1

k += 1

# Copy remaining elements of left array
while i < len(left):
arr[k] = left[i]
i += 1
k += 1

# Copy remaining elements of right array
while j < len(right):
arr[k] = right[j]
j += 1
k += 1

# Main Program
```

```
n = int(input("Enter number of elements: "))

arr = []

print("Enter elements:")
for i in range(n):
arr.append(int(input()))

merge_sort(arr)

print("Sorted Array:")
for element in arr:
    print(element, end=" ")
```

OUTPUT:



EXPLANATION:

1. Divide the array into smaller parts.

2. Continue dividing until single elements remain.

3. Merge the elements in sorted order.

4. Uses Divide and Conquer.

5. Time Complexity: O(n log n)

4(B).AIM: Write a python program to implement quick sort where the input elements are taken from the user

PROGRAM:

```
def quicksort(a, low, high):
    if low < high:
        i = low
        j = high
        pivot = low
 # Poojasree av.sc.u4cse25103
        while i < j:

            # Move i from left to right
            while i <= high and a[i] <= a[pivot]:
                i += 1

            # Move j from right to left
            while a[j] > a[pivot]:
                j -= 1

            # Swap elements if pointers have not crossed
            if i < j:
                a[i], a[j] = a[j], a[i]

        # Place pivot in its correct position
        a[j], a[pivot] = a[pivot], a[j]

        # Sort left part
        quicksort(a, low, j - 1)

        # Sort right part
        quicksort(a, j + 1, high)

# Main Program
a = list(map(int, input("Enter numbers to sort: ").split()))

n = len(a)

quicksort(a, 0, n - 1)

print("Sorted array:")
print(a)
```

OUTPUT:



EXPLANATION:

1.Select an element as the pivot.

2.Partition the array around the pivot.

3.Smaller elements go to the left and larger elements to the right.

4.Recursively sort the left and right parts.

5.Uses Divide and Conquer.

6.Average Time Complexity: O(n log n).

WEEK-5

5(A).AIM: Write a python program to implement a single linked list with the below operations
i)Create linked list
ii)insert beginning
iii)insert end
iv)insert at index
v) delete by value
vi)delete first node
vii)delete last node
viii)count number of nodes
ix)display/traverse
x)exit

PROGRAM:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

#poojasree av.sc.u4cse25103
class SinglyLinkedList:

    def __init__(self):
        self.head = None

    # i) Create linked list
    def create(self):
        n = int(input("Enter number of elements: "))

        for i in range(n):
            data = int(input("Enter value: "))
            new = Node(data)

            if self.head is None:
                self.head = new
            else:
                temp = self.head

                while temp.next:
                    temp = temp.next

                temp.next = new
```

```
# ii) Insert at beginning
def insert_begin(self, data):
    new = Node(data)
    new.next = self.head
    self.head = new

# iii) Insert at end
def insert_end(self, data):
    new = Node(data)

    if self.head is None:
        self.head = new
    else:
        temp = self.head

        while temp.next:
            temp = temp.next

        temp.next = new

# iv) Insert at index
def insert_at_index(self, data, index):
    if index < 0:
        print("Invalid index")
        return

    if index == 0:
        self.insert_begin(data)
        return

    new = Node(data)
    temp = self.head

    for i in range(index - 1):
        if temp is None:
            print("Index out of range")
            return
        temp = temp.next

    if temp is None:
        print("Index out of range")
        return

    new.next = temp.next
    temp.next = new
```

```
# v) Delete by value
def delete_by_value(self, value):
    if self.head is None:
        print("List is empty")
        return

    if self.head.data == value:
        self.head = self.head.next
        print("Deleted value =", value)
        return

    temp = self.head

    while temp.next and temp.next.data != value:
        temp = temp.next

    if temp.next is None:
        print("Value not found")
    else:
        temp.next = temp.next.next
        print("Deleted value =", value)

# vi) Delete first node
def delete_first(self):
    if self.head is None:
        print("No Data to delete")
    else:
        temp = self.head
        self.head = self.head.next
        print("Deleted value =", temp.data)

# vii) Delete last node
def delete_last(self):
    if self.head is None:
        print("No Data to delete")
        return

    if self.head.next is None:
        print("Deleted value =", self.head.data)
        self.head = None
        return

    temp = self.head

    while temp.next.next:
        temp = temp.next
```

```
            print("Deleted value =", temp.next.data)
            temp.next = None

        # viii) Count number of nodes
        def count_nodes(self):
            count = 0
            temp = self.head

            while temp:
                count += 1
                temp = temp.next

            print("Number of nodes =", count)

        # ix) Display / Traverse
        def display(self):
            if self.head is None:
                print("List is empty")
            else:
                temp = self.head

                while temp:
                    print(temp.data, end=" ")
                    temp = temp.next

                print()

    # Main Program
    s = SinglyLinkedList()

    while True:

        print("\n----- SINGLY LINKED LIST -----")
        print("1. Create linked list")
        print("2. Insert at beginning")
        print("3. Insert at end")
        print("4. Insert at index")
        print("5. Delete by value")
        print("6. Delete first node")
        print("7. Delete last node")
        print("8. Count number of nodes")
        print("9. Display / Traverse")
        print("10. Exit")

        choice = int(input("Enter your choice: "))
```

```
if choice == 1:
    s.create()

elif choice == 2:
    data = int(input("Enter value: "))
    s.insert_begin(data)

elif choice == 3:
    data = int(input("Enter value: "))
    s.insert_end(data)

elif choice == 4:
    data = int(input("Enter value: "))
    index = int(input("Enter index: "))
    s.insert_at_index(data, index)

elif choice == 5:
    value = int(input("Enter value to delete: "))
    s.delete_by_value(value)

elif choice == 6:
    s.delete_first()

elif choice == 7:
    s.delete_last()

elif choice == 8:
    s.count_nodes()

elif choice == 9:
    s.display()

elif choice == 10:
    print("Exiting...")
    break

else:
    print("Invalid choice")
```

OUTPUT:



EXPLANATION:

1. Each node contains data and a next pointer.

2. The next pointer stores the address of the next node.

3.The last node points to NULL.

4. Traversal is possible in one direction.

5. Insertion and deletion can be performed at different positions.

ERROR TABLE:

ERRORS	RECTIFICATION
Empty list error occurred when an operation was performed on an empty list.	Checked self.head is None before performing the operation.
Node Error occurred because the Node class was not defined properly.	Defined the Node class before using it.

5(B).AIM. Write a python program to implement a double linked list with the below operations
i)Create linked list
ii)insert beginning
iii)insert end
iv)insert at index
v) delete by value
vi)delete first node
vii)delete last node
viii)count number of nodes
ix)display/traverse
x)exit

PROGRAM:

```
#POOJASREE AV.SC.U4CSE25103
class Node:
    def __init__(self, data):
        self.data = data
        self.prev = None
        self.next = None

#poojasree av.sc.u4cse25103
class DoublyLinkedList:

    def __init__(self):
        self.head = None

    # i) Create linked list
    def create(self):
        n = int(input("Enter number of elements: "))

        for i in range(n):
            data = int(input("Enter value: "))
            new = Node(data)

            if self.head is None:
                self.head = new
            else:
                temp = self.head

                while temp.next:
                    temp = temp.next

                temp.next = new
                new.prev = temp

    # ii) Insert at beginning
```

```
def insert_begin(self, data):
    new = Node(data)

    if self.head is None:
        self.head = new
    else:
        new.next = self.head
        self.head.prev = new
        self.head = new

# iii) Insert at end
def insert_end(self, data):
    new = Node(data)

    if self.head is None:
        self.head = new
    else:
        temp = self.head

        while temp.next:
            temp = temp.next

        temp.next = new
        new.prev = temp

# iv) Insert at index
def insert_at_index(self, data, index):
    if index < 0:
        print("Invalid index")
        return

    if index == 0:
        self.insert_begin(data)
        return

    temp = self.head

    for i in range(index - 1):
        if temp is None:
            print("Index out of range")
            return
        temp = temp.next

    if temp is None:
        print("Index out of range")
        return
```

```
        new = Node(data)

        new.next = temp.next
        new.prev = temp

        if temp.next is not None:
            temp.next.prev = new

        temp.next = new

    # v) Delete by value
    def delete_by_value(self, value):
        if self.head is None:
            print("List is empty")
            return

        temp = self.head

        while temp is not None and temp.data != value:
            temp = temp.next

        if temp is None:
            print("Value not found")
            return

        if temp.prev is not None:
            temp.prev.next = temp.next
        else:
            self.head = temp.next

        if temp.next is not None:
            temp.next.prev = temp.prev

        print("Deleted value =", value)

    # vi) Delete first node
    def delete_first(self):
        if self.head is None:
            print("No Data to delete")
        else:
            temp = self.head
            self.head = self.head.next

            if self.head is not None:
                self.head.prev = None
```

```
            print("Deleted value =", temp.data)

    # vii) Delete last node
    def delete_last(self):
        if self.head is None:
            print("No Data to delete")
            return

        temp = self.head

        while temp.next:
            temp = temp.next

        if temp.prev is None:
            self.head = None
        else:
            temp.prev.next = None

        print("Deleted value =", temp.data)

    # viii) Count number of nodes
    def count_nodes(self):
        count = 0
        temp = self.head

        while temp:
            count += 1
            temp = temp.next

        print("Number of nodes =", count)

    # ix) Display / Traverse
    def display(self):
        if self.head is None:
            print("List is empty")
        else:
            temp = self.head

            while temp:
                print(temp.data, end=" ")
                temp = temp.next

            print()

# Main Program
```

```
d = DoublyLinkedList()

while True:

    print("\n----- DOUBLY LINKED LIST -----")
    print("1. Create linked list")
    print("2. Insert at beginning")
    print("3. Insert at end")
    print("4. Insert at index")
    print("5. Delete by value")
    print("6. Delete first node")
    print("7. Delete last node")
    print("8. Count number of nodes")
    print("9. Display / Traverse")
    print("10. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        d.create()

    elif choice == 2:
        data = int(input("Enter value: "))
        d.insert_begin(data)

    elif choice == 3:
        data = int(input("Enter value: "))
        d.insert_end(data)

    elif choice == 4:
        data = int(input("Enter value: "))
        index = int(input("Enter index: "))
        d.insert_at_index(data, index)

    elif choice == 5:
        value = int(input("Enter value to delete: "))
        d.delete_by_value(value)

    elif choice == 6:
        d.delete_first()

    elif choice == 7:
        d.delete_last()

    elif choice == 8:
        d.count_nodes()
```

```
    elif choice == 9:
        d.display()

    elif choice == 10:
        print("Exiting...")
        break

    else:
        print("Invalid choice")
```

OUTPUT:



EXPLANATION:

1.Each node contains data, previous pointer, and next pointer.

2.previous pointer points to the previous node.

3.The next pointer points to the next node.

4.Traversal is possible in both directions.

5.The first node's previous pointer and last node's next pointer are NULL.

ERROR TABLE:

ERRORS	RECTIFICATION

1.IndentationError occurred because the statements were not properly aligned.	Corrected the indentation of all blocks.
2.NameError occurred because new_node, temp, or other variables were misspelled.	Corrected the variable names and spelling.

5(C).AIM. Write a python program to implement a circular linked list with the below operations
i)Create linked list
ii)insert beginning
iii)insert end
iv)insert at index
v) delete by value
vi)delete first node
vii)delete last node
viii)count number of nodes
ix)display/traverse
x)exit

PROGRAM:

```
#POOJASREE AV.SC.U4CSE25103
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
#poojasree av.sc.u4cse25103

class CircularLinkedList:

    def __init__(self):
        self.head = None

    # i) Create linked list
    def create(self):
        n = int(input("Enter number of elements: "))

        for i in range(n):
            data = int(input("Enter value: "))
            new = Node(data)

            if self.head is None:
                self.head = new
```

```
            new.next = self.head
        else:
            temp = self.head

            while temp.next != self.head:
                temp = temp.next

            temp.next = new
            new.next = self.head

# ii) Insert at beginning
def insert_begin(self, data):
    new = Node(data)

    if self.head is None:
        self.head = new
        new.next = self.head
    else:
        temp = self.head

        while temp.next != self.head:
            temp = temp.next

        new.next = self.head
        temp.next = new
        self.head = new

# iii) Insert at end
def insert_end(self, data):
    new = Node(data)

    if self.head is None:
        self.head = new
        new.next = self.head
    else:
        temp = self.head

        while temp.next != self.head:
            temp = temp.next

        temp.next = new
        new.next = self.head

# iv) Insert at index
def insert_at_index(self, data, index):
    if index < 0:
```

```
            print("Invalid index")
            return

        if index == 0:
            self.insert_begin(data)
            return

        if self.head is None:
            print("Index out of range")
            return

        temp = self.head

        for i in range(index - 1):
            temp = temp.next

            if temp == self.head:
                print("Index out of range")
                return

        new = Node(data)
        new.next = temp.next
        temp.next = new

    # v) Delete by value
    def delete_by_value(self, value):
        if self.head is None:
            print("List is empty")
            return

        # If only one node exists
        if self.head.data == value and self.head.next == self.head:
            self.head = None
            print("Deleted value =", value)
            return

        # If first node contains the value
        if self.head.data == value:
            self.delete_first()
            return

        temp = self.head

        while temp.next != self.head and temp.next.data != value:
            temp = temp.next
```

```
        if temp.next == self.head:
            print("Value not found")
        else:
            temp.next = temp.next.next
            print("Deleted value =", value)

    # vi) Delete first node
    def delete_first(self):
        if self.head is None:
            print("No Data to delete")
            return

        # Only one node
        if self.head.next == self.head:
            print("Deleted value =", self.head.data)
            self.head = None
            return

        temp = self.head

        while temp.next != self.head:
            temp = temp.next

        print("Deleted value =", self.head.data)

        self.head = self.head.next
        temp.next = self.head

    # vii) Delete last node
    def delete_last(self):
        if self.head is None:
            print("No Data to delete")
            return

        # Only one node
        if self.head.next == self.head:
            print("Deleted value =", self.head.data)
            self.head = None
            return

        temp = self.head

        while temp.next.next != self.head:
            temp = temp.next

        print("Deleted value =", temp.next.data)
```

```
            temp.next = self.head

        # viii) Count number of nodes
        def count_nodes(self):
            if self.head is None:
                print("Number of nodes = 0")
                return

            count = 0
            temp = self.head

            while True:
                count += 1
                temp = temp.next

                if temp == self.head:
                    break

            print("Number of nodes =", count)

        # ix) Display / Traverse
        def display(self):
            if self.head is None:
                print("List is empty")
                return

            temp = self.head

            while True:
                print(temp.data, end=" ")
                temp = temp.next

                if temp == self.head:
                    break

            print()

    # Main Program
    c = CircularLinkedList()

    while True:

        print("\n----- CIRCULAR LINKED LIST -----")
        print("1. Create linked list")
        print("2. Insert at beginning")
```

```
print("3. Insert at end")
print("4. Insert at index")
print("5. Delete by value")
print("6. Delete first node")
print("7. Delete last node")
print("8. Count number of nodes")
print("9. Display / Traverse")
print("10. Exit")

choice = int(input("Enter your choice: "))

if choice == 1:
    c.create()

elif choice == 2:
    data = int(input("Enter value: "))
    c.insert_begin(data)

elif choice == 3:
    data = int(input("Enter value: "))
    c.insert_end(data)

elif choice == 4:
    data = int(input("Enter value: "))
    index = int(input("Enter index: "))
    c.insert_at_index(data, index)

elif choice == 5:
    value = int(input("Enter value to delete: "))
    c.delete_by_value(value)

elif choice == 6:
    c.delete_first()

elif choice == 7:
    c.delete_last()

elif choice == 8:
    c.count_nodes()

elif choice == 9:
    c.display()

elif choice == 10:
    print("Exiting...")
    break
```

```
    else:
        print("Invalid choice")
```

OUTPUT:



EXPLANATION:

1.Each node contains data and a next pointer.

2.The last node points back to the first node.

3.There is no NULL pointer at the end.

4.Traversal can continue in a circular manner.

5.It can be implemented as a singly or doubly circular list.

ERROR TABLE:

ERRORS	RECTIFICATION
1. AttributeError occurred when the next link was not updated properly.	Updated the next links correctly to maintain the circular connection.
2. IndexError occurred when an invalid index was entered.	Checked whether the index is within the valid range.

WEEK-6

6(A).AIM.Write a python program to implement stack operations such as push, pop, peek, display using arrays

PROGRAM:

```
#POOJASREE AV.SC.U4CSE25103
class Stack:

    def __init__(self, size):
        self.size = size
        self.stack = [None] * size
        self.top = -1
#poojasree av.sc.u4cse25103
    # Push
    def push(self, data):
        if self.top == self.size - 1:
            print("Stack Overflow")
        else:
            self.top += 1
            self.stack[self.top] = data

    # Pop
    def pop(self):
        if self.top == -1:
            print("Stack Underflow")
        else:
            data = self.stack[self.top]
            self.top -= 1
            print("Deleted value =", data)

    # Peek
    def peek(self):
        if self.top == -1:
            print("Stack Underflow")
        else:
            print("Top element =", self.stack[self.top])

    # Display
    def display(self):
        if self.top == -1:
            print("Stack is empty")
        else:
```

```
            for i in range(self.top, -1, -1):
                print(self.stack[i], end=" ")
            print()

# Main Program
size = int(input("Enter size of stack: "))
s = Stack(size)

while True:

    print("\n1. Push")
    print("2. Pop")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter choice: "))

    if choice == 1:
        data = int(input("Enter value: "))
        s.push(data)

    elif choice == 2:
        s.pop()

    elif choice == 3:
        s.peek()

    elif choice == 4:
        s.display()

    elif choice == 5:
        break

    else:
        print("Invalid choice")
```

OUTPUT:



EXPLANATION:

1. Stack is implemented using an array to store elements.

2. A variable called TOP keeps track of the top element.

3. Initially, TOP = -1.

4. Push adds an element by increasing TOP.

5. Pop removes the element at TOP and decreases TOP.

6. Stack Overflow occurs when the array is full.
7. Stack Underflow occurs when the stack is empty.

ERROR TABLE:

ERRORS	RECTIFICATION
1. ValueError occurred when a noninteger value was entered.	Entered only valid integer values.
2.Index Error occurred when trying to access the top element of an empty stack.	Checked whether the stack is empty before accessing an element.

6(B).AIM. Write a python program to implement stack operations such as push, pop, peek, display using linked lists.

PROGRAM:

```
#POOJASREE AV.SC.U4CSE25103
class Node:

    def __init__(self, data):
        self.data = data
        self.next = None

#poojasree av.sc.u4cse25103
class Stack:

    def __init__(self):
        self.top = None

    # Push
    def push(self, data):
        new = Node(data)
        new.next = self.top
        self.top = new

    # Pop
    def pop(self):
        if self.top is None:
            print("Stack Underflow")
        else:
            temp = self.top
            print("Deleted value =", temp.data)
            self.top = self.top.next

    # Peek
    def peek(self):
        if self.top is None:
            print("Stack Underflow")
        else:
            print("Top element =", self.top.data)

    # Display
    def display(self):
        if self.top is None:
            print("Stack is empty")
        else:
            temp = self.top

            while temp:
                print(temp.data, end=" ")
                temp = temp.next
```

```
        print()

# Main Program
s = Stack()

while True:

    print("\n1. Push")
    print("2. Pop")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter choice: "))

    if choice == 1:
        data = int(input("Enter value: "))
        s.push(data)

    elif choice == 2:
        s.pop()

    elif choice == 3:
        s.peek()

    elif choice == 4:
        s.display()

    elif choice == 5:
        break

    else:
        print("Invalid choice")
```

OUTPUT:



EXPLANATION:

1.Stack is implemented using a linked list of nodes.

2.Each node contains data and a next pointer.

3.A pointer called TOP points to the top node

4.Push adds a new node at the beginning.

5.Pop removes the node pointed to by TOP.

6.It does not have a fixed size like an array.

7.Stack Underflow occurs when TOP is NULL.

ERROR TABLE:

ERROR	RECTIFICATION
1.Empty stack error occurred when Pop or Peek was performed without any elements.	Checked self.top is None before performing the operation.

WEEK-7

7(A).AIM. Write a program to implement queue and its operations such as enqueue, dequeue,peek,display with a menu driven approach using arrays

PROGRAM:

```
#POOJASREE AV.SC.U4CSE25103
class Queue:
    def __init__(self, size):
        self.size = size
        self.queue = [0] * size
        self.front = -1
        self.rear = -1

    def enqueue(self, data):
        if self.rear == self.size - 1:
            print("Queue Overflow")
        else:
            if self.front == -1:
                self.front = 0
            self.rear += 1
            self.queue[self.rear] = data
            print(data, "inserted")

    def dequeue(self):
        if self.front == -1 or self.front > self.rear:
            print("Queue Underflow")
        else:
            print(self.queue[self.front], "deleted")
            self.front += 1

    def peek(self):
        if self.front == -1 or self.front > self.rear:
            print("Queue is empty")
        else:
            print("Front element:", self.queue[self.front])

    def display(self):
        if self.front == -1 or self.front > self.rear:
            print("Queue is empty")
        else:
            for i in range(self.front, self.rear + 1):
                print(self.queue[i], end=" ")
            print()

size = int(input("Enter queue size: "))
```

```
q = Queue(size)

while True:
    print("\n1.Enqueue  2.Dequeue  3.Peek  4.Display  5.Exit")
    choice = int(input("Enter choice: "))

    if choice == 1:
        data = int(input("Enter element: "))
        q.enqueue(data)
    elif choice == 2:
        q.dequeue()
    elif choice == 3:
        q.peek()
    elif choice == 4:
        q.display()
    elif choice == 5:
        break
    else:
        print("Invalid choice")
```

OUTPUT:



EXPLANTION:

1. Queue is implemented using an array.
2. Two variables, FRONT and REAR, are used.
3. FRONT points to the first element.
4. REAR points to the last element.
5. Enqueue adds an element at the REAR.
6. Dequeue removes an element from the FRONT.
7. Peek displays the front element.

8. Display shows all elements from FRONT to REAR.
9. Queue Overflow occurs when the array is full.
10. Queue Underflow occurs when the queue is empty.

ERROR TABLE:

ERROR	RECTIFICATION
1.list assignment index out of range occurred when an element was inserted beyond the array size.	Checked the queue overflow condition before insertion.
2.Queue Underflow occurred when Dequeue or Peek was performed on an empty queue.	Checked self.front == -1 or self.front > self.rear before the operation.

7(B).AIM. Write a program to implement queue and its operations such as enqueue, dequeue,peek,display with a menu driven approach using linked lists

PROGRAM:

```
#POOJASREE AV.SC.U4CSE25103
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class Queue:
    def __init__(self):
        self.front = None
        self.rear = None

    def enqueue(self, data):
        new = Node(data)

        if self.rear is None:
            self.front = self.rear = new
        else:
            self.rear.next = new
            self.rear = new

        print(data, "inserted")

    def dequeue(self):
        if self.front is None:
```

```
            print("Queue Underflow")
        else:
            print(self.front.data, "deleted")
            self.front = self.front.next

            if self.front is None:
                self.rear = None

    def peek(self):
        if self.front is None:
            print("Queue is empty")
        else:
            print("Front element:", self.front.data)

    def display(self):
        if self.front is None:
            print("Queue is empty")
        else:
            temp = self.front
            while temp is not None:
                print(temp.data, end=" ")
                temp = temp.next
            print()

q = Queue()

while True:
    print("\n1.Enqueue 2.Dequeue 3.Peek 4.Display 5.Exit")
    choice = int(input("Enter choice: "))

    if choice == 1:
        data = int(input("Enter element: "))
        q.enqueue(data)
    elif choice == 2:
        q.dequeue()
    elif choice == 3:
        q.peek()
    elif choice == 4:
        q.display()
    elif choice == 5:
        break
    else:
        print("Invalid choice")
```

OUTPUT:



EXPLANATION:

1. Queue is implemented using linked-list nodes.
2. Each node contains data and a next pointer.
3. Two pointers, FRONT and REAR, are used.
4. Enqueue adds a new node at the REAR.
5. Dequeue removes a node from the FRONT.
6. Peek displays the front element.
7. Display traverses the linked list from FRONT to REAR.
8. It does not have a fixed size like an array.
9. Underflow occurs when the queue is empty.

ERROR TABLE:

ERROR	RECTIFICATION
1.name 'Node' is not defined occurred when the Node class was not defined or was misspelled.	Defined the Node class correctly before using it.

7(C).AIM. Write a program to implement circular queue and its operations such as enqueue, dequeue,peek,display with a menu driven approach using arrays

PROGRAM:

```
#POOJASREE AV.SC.U4CSE25103
class CircularQueue:
    def __init__(self, size):
        self.size = size
        self.queue = [0] * size
        self.front = -1
        self.rear = -1

    def enqueue(self, data):
        if (self.rear + 1) % self.size == self.front:
            print("Queue Overflow")
        else:
            if self.front == -1:
                self.front = 0
                self.rear = 0
            else:
                self.rear = (self.rear + 1) % self.size

            self.queue[self.rear] = data
            print(data, "inserted")

    def dequeue(self):
        if self.front == -1:
            print("Queue Underflow")
        else:
            print(self.queue[self.front], "deleted")

            if self.front == self.rear:
                self.front = self.rear = -1
            else:
                self.front = (self.front + 1) % self.size

    def peek(self):
        if self.front == -1:
            print("Queue is empty")
        else:
            print("Front element:", self.queue[self.front])

    def display(self):
        if self.front == -1:
            print("Queue is empty")
        else:
            i = self.front

            while True:
```

```
            print(self.queue[i], end=" ")

            if i == self.rear:
                break

            i = (i + 1) % self.size

        print()

size = int(input("Enter queue size: "))
q = CircularQueue(size)

while True:
    print("\n1.Enqueue  2.Dequeue  3.Peek  4.Display  5.Exit")
    choice = int(input("Enter choice: "))

    if choice == 1:
        data = int(input("Enter element: "))
        q.enqueue(data)
    elif choice == 2:
        q.dequeue()
    elif choice == 3:
        q.peek()
    elif choice == 4:
        q.display()
    elif choice == 5:
        break
    else:
        print("Invalid choice")
```

OUTPUT:



EXPLANATION:

1. Circular queue is implemented using an array.
2. It uses FRONT and REAR to track elements.

3. Enqueue adds an element at REAR.
4. Dequeue removes an element from FRONT.
5. When REAR reaches the last position, it wraps around to the beginning.
6. The modulo operator % is used for circular movement.
7. Peek displays the front element.
8. Display shows all elements in circular order.
9. It makes better use of the available array space.

ERROR TABLE:

ERROR	RECTIFICATION
1.name 'rear' is not defined occurred when rear was used instead of self.rear.	Corrected the variable name to self.rear.
2. 'CircularQueue' object has no attribute 'front' occurred when self.front was not initialized.	Initialized self.front = -1 in the constructor.

7(D).AIM: Write a program to implement circular queue and its operations such as enqueue, dequeue,peek,display with a menu driven approach using linked lists.

PROGRAM:

```
#POOJASREE AV.SC.U4CSE25103
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class CircularQueue:
    def __init__(self):
        self.front = None
        self.rear = None

    def enqueue(self, data):
        new = Node(data)
```

```
        if self.front is None:
            self.front = self.rear = new
            new.next = self.front
        else:
            new.next = self.front
            self.rear.next = new
            self.rear = new

        print(data, "inserted")

    def dequeue(self):
        if self.front is None:
            print("Queue Underflow")
        else:
            print(self.front.data, "deleted")

            if self.front == self.rear:
                self.front = self.rear = None
            else:
                self.front = self.front.next
                self.rear.next = self.front

    def peek(self):
        if self.front is None:
            print("Queue is empty")
        else:
            print("Front element:", self.front.data)

    def display(self):
        if self.front is None:
            print("Queue is empty")
        else:
            temp = self.front

            while True:
                print(temp.data, end=" ")

                temp = temp.next

                if temp == self.front:
                    break

            print()

q = CircularQueue()

while True:
    print("\n1.Enqueue  2.Dequeue  3.Peek  4.Display  5.Exit")
    choice = int(input("Enter choice: "))

    if choice == 1:
```

```
        data = int(input("Enter element: "))
        q.enqueue(data)
    elif choice == 2:
        q.dequeue()
    elif choice == 3:
        q.peek()
    elif choice == 4:
        q.display()
    elif choice == 5:
        break
    else:
        print("Invalid choice")
```

OUTPUT:



EXPLANATION:

1. Circular queue is implemented using a linked list.
2. Each node contains data and a next pointer.
3. The REAR node points back to the FRONT node.
4. Enqueue adds a new node at REAR.
5. Dequeue removes a node from FRONT.
6. Peek displays the front element.
7. Display traverses the nodes until it reaches FRONT again.
8. It does not require a fixed array size.
9. The queue maintains a circular connection between REAR and FRONT.

ERROR TABLE:

ERROR	RECTIFICATION
1.'NoneType' object has no attribute 'next' occurred when the rear node was accessed before creating a queue node.	Checked whether the queue was empty before accessing
2. invalid literal for int() with base 10 occurred when a non-integer value was entered.	Entered only valid integer values

WEEK-8

8(A).Aim: Write a python program to implement a binary tree with its operations using arrays

PROGRAM:

```
#POOJASREE AV.SC.U4CSE25103
class BinaryTree:
    def __init__(self, size):
        self.tree = [None] * size
        self.size = size

    def insert(self, data, index=0):
        if index >= self.size:
            print("Tree is full")
            return

        if self.tree[index] is None:
            self.tree[index] = data
        else:
            print("Position occupied")

    def preorder(self, index=0):
        if index < self.size and self.tree[index] is not None:
            print(self.tree[index], end=" ")
            self.preorder(2 * index + 1)
            self.preorder(2 * index + 2)

    def inorder(self, index=0):
        if index < self.size and self.tree[index] is not None:
            self.inorder(2 * index + 1)
            print(self.tree[index], end=" ")
            self.inorder(2 * index + 2)

    def postorder(self, index=0):
        if index < self.size and self.tree[index] is not None:
            self.postorder(2 * index + 1)
            self.postorder(2 * index + 2)
            print(self.tree[index], end=" ")

    def display(self):
        print(self.tree)


bt = BinaryTree(7)
```

```
bt.insert(10, 0)
bt.insert(20, 1)
bt.insert(30, 2)
bt.insert(40, 3)
bt.insert(50, 4)

print("Array:")
bt.display()

print("Preorder:")
bt.preorder()

print("\nInorder:")
bt.inorder()

print("\nPostorder:")
bt.postorder()
```

OUTPUT:



EXPLANATION:

1. __init__(size) – Creates an array of the given size and initializes all positions with None.

2. insert(data, index) – Inserts data at the specified index if the position is empty; otherwise, displays a message.

3. preorder(index) – Traverses the tree in the order Root → Left → Right.

4. inorder(index) – Traverses the tree in the order Left → Root → Right.

5. postorder(index) – Traverses the tree in the order Left → Right → Root.

6. display() – Displays the tree as an array.

7. 2 * index + 1 – Calculates the left child's index.

8. 2 * index + 2 – Calculates the right child's index.

9. bt = BinaryTree(7) – Creates a binary tree with 7 positions.

10 Function calls – Insert five elements and display the array and all three traversals.

ERROR TABLE:

ERROR	RECTIFICATION
1. AttributeError: 'BinaryTree' object has no attribute 'preorder'	Defined the preorder() method inside the class with proper indentation.
2.TypeError: BinaryTree() takes no arguments	Corrected the constructor name to __init__.

8(B). AIM: Write a python program to implement a binary tree with its operations using linked lists

PROGRAM:

```
#POOJASREE AV.SC.U4CSE25103

class Node:

    def __init__(self, data):

        self.data = data

        self.left = None

        self.right = None

 def insert(root, data):

    new_node = Node(data)

    if root is None:

        return new_node

    queue = [root]
```

```
    while queue:
        temp = queue.pop(0)
        if temp.left is None:
            temp.left = new_node
            return root
        else:
            queue.append(temp.left)
        if temp.right is None:
            temp.right = new_node
            return root
else:
            queue.append(temp.right)

def preorder(root):
    if root:
        print(root.data, end=" ")
        preorder(root.left)
        preorder(root.right)
 def inorder(root):
    if root:
        inorder(root.left)
        print(root.data, end=" ")
        inorder(root.right)
 def postorder(root):
```

```
    if root:
        postorder(root.left)
        postorder(root.right)
        print(root.data, end=" ")
root = None
for x in [10, 20, 30, 40, 50]:
    root = insert(root, x)
print("Preorder:")
preorder(root)
print("\nInorder:")
inorder(root)
print("\nPostorder:")
postorder(root)
```

OUTPUT:



EXPLANATION:

1.class Node – Creates a node for the binary tree.

2. _init__(data) – Initializes the node with data and sets left and right to None.

3. insert(root, data) – Inserts a new node at the first available position, level by level.

4. if root is None – Creates the root node if the tree is empty.

5. queue = [root] – Creates a queue to traverse the tree level by level.

6. queue.pop(0) – Removes and returns the first node from the queue.

7. temp.left = new_node – Inserts the new node as the left child if the position is empty.

8. temp.right = new_node – Inserts the new node as the right child if the position is empty.

9. preorder(root) – Traverses in the order Root → Left → Right.

10. inorder(root) – Traverses in the order Left → Root → Right.

11. postorder(root) – Traverses in the order Left → Right → Root.
12. for x in [10, 20, 30, 40, 50] – Inserts five elements into the binary tree.

13. root = insert(root, x) – Updates the root after each insertion.

ERROR TABLE:

ERROR	RECTIFICATION
IndentationError: expected an indented block	Use proper 4-space indentation inside functions and conditions.

WEEK-9

9(A).AIM: Write a python program to implement a bst with its operations using linked list

PROGRAM:

```
#POOJASREE AV.SC.U4CSE25103
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None
def insert(root, data):
    if root is None:
        return Node(data)
    if data < root.data:
        root.left = insert(root.left, data)
    elif data > root.data:
        root.right = insert(root.right, data)
    return root
def search(root, key):
    if root is None or root.data == key:
        return root
    if key < root.data:
        return search(root.left, key)
    return search(root.right, key)
def inorder(root):
    if root:
        inorder(root.left)
        print(root.data, end=" ")
        inorder(root.right)
def preorder(root):
    if root:
        print(root.data, end=" ")
        preorder(root.left)
        preorder(root.right)
def postorder(root):
    if root:
        postorder(root.left)
        postorder(root.right)
        print(root.data, end=" ")
def delete(root, key):
    if root is None:
        return None
   if key < root.data:
        root.left = delete(root.left, key)
    elif key > root.data:
```

```
        root.right = delete(root.right, key)
    else:
        if root.left is None:
            return root.right
        if root.right is None:
            return root.left

        temp = root.right
        while temp.left:
            temp = temp.left

        root.data = temp.data
        root.right = delete(root.right, temp.data)

    return root
root = None
while True:
    print("\n1.Insert  2.Search  3.Delete")
    print("4.Inorder  5.Preorder  6.Postorder  7.Exit")
    ch = int(input("Enter choice: "))

    if ch == 1:
        x = int(input("Enter value: "))
        root = insert(root, x)
    elif ch == 2:
        x = int(input("Enter value to search: "))
        print("Found" if search(root, x) else "Not Found")
    elif ch == 3:
        x = int(input("Enter value to delete: "))
        root = delete(root, x)
    elif ch == 4:
        inorder(root)
        print()
    elif ch == 5:
        preorder(root)
        print()
    elif ch == 6:
        postorder(root)
        print()
    elif ch == 7:
        break
    else:
        print("Invalid choice")
```

OUTPUT:



EXPLANATION:

1. class Node – Creates a node for the BST.

2. __init__(data) – Initializes the node with data and sets left and right children to None.

3. insert(root, data) – Inserts a value into the BST. Smaller values go left, and larger values go right.

4. search(root, key) – Searches for a value in the BST and returns the node if found.

5. inorder(root) – Traverses the tree in the order Left → Root → Right.

6. preorder(root) – Traverses the tree in the order Root → Left → Right.

7. postorder(root) – Traverses the tree in the order Left → Right → Root.

8. delete(root, key) – Deletes a node from the BST while maintaining its structure and ordering.

9. if root.left is None – Handles deletion when the node has no left child.

10. if root.right is None – Handles deletion when the node has no right child.

11. while temp.left – Finds the inorder successor (smallest value in the right subtree) during deletion.

12. root = None – Initializes an empty BST.

13. while True – Repeats the menu until the user chooses Exit.

14. ch = int(input(...)) – Accepts the user's menu choice.

15. if-elif conditions – Execute insertion, search, deletion, traversal, or exit based on the selected option.

ERROR TABLE:

ERROR	RECTIFICATION
RecursionError: maximum recursion depth exceeded	Avoid extremely deep trees or use an iterative approach for large inputs.

9(B).AIM: Write a python program to implement a bst with its operations using arrays

PROGRAM:

```
#POOJASREE AV.SC.U4CS.U4CSE25103
tree = []
def insert(x):
    if not tree:
        tree.append(x)
        return

    i = 0
    while i < len(tree):
        if tree[i] is None:
            tree[i] = x
            return
        if x == tree[i]:
            return
        if x < tree[i]:
            i = 2 * i + 1
        else:
            i = 2 * i + 2

    tree.extend([None] * (i - len(tree) + 1))
    tree[i] = x
def search(x):
    i = 0
    while i < len(tree) and tree[i] is not None:
        if tree[i] == x:
            return i
        if x < tree[i]:
            i = 2 * i + 1
```

```
        else:
            i = 2 * i + 2
    return -1
def inorder(i=0):
    if i < len(tree) and tree[i] is not None:
        inorder(2 * i + 1)
        print(tree[i], end=" ")
        inorder(2 * i + 2)
def preorder(i=0):
    if i < len(tree) and tree[i] is not None:
        print(tree[i], end=" ")
        preorder(2 * i + 1)
        preorder(2 * i + 2)
def postorder(i=0):
    if i < len(tree) and tree[i] is not None:
        postorder(2 * i + 1)
        postorder(2 * i + 2)
        print(tree[i], end=" ")
def delete(x):
    i = search(x)
    if i == -1:
        print("Not Found")
        return

    # Collect all values except the deleted value.
    values = [
        v for v in tree if v is not None and v != x
    ]

    tree.clear()
    for v in values:
        insert(v)
while True:
    print("\n1.Insert  2.Search  3.Delete")
    print("4.Inorder  5.Preorder  6.Postorder  7.Exit")
    ch = int(input("Enter choice: "))
    if ch == 1:
        insert(int(input("Enter value: ")))
    elif ch == 2:
        x = int(input("Enter value to search: "))
        print("Found" if search(x) != -1 else "Not Found")
    elif ch == 3:
        delete(int(input("Enter value to delete: ")))
    elif ch == 4:
        inorder()
        print()
```

```
elif ch == 5:
    preorder()
    print()
elif ch == 6:
    postorder()
    print()
elif ch == 7:
    break
else:
    print("Invalid choice")
```

OUTPUT:



ERROR TABLE:

ERROR	RECTIFICATION
1.RecursionError: maximum recursion depth exceeded	Avoid extremely deep trees or use an iterative approach for large inputs.
2.NameError: name 'insert' is not defined	Define the insert() function before calling it.

EXPLANATION:

1. tree = [] – Creates an empty array to store the BST elements.

2. insert(x) – Inserts a value into the BST, placing smaller values on the left and larger values on the right.

3. tree.extend() – Expands the array with None values when more space is required.

4. search(x) – Searches for a value and returns its index if found, otherwise returns -1.

5. inorder(i) – Traverses the tree in the order Left → Root → Right.

6. preorder(i) – Traverses the tree in the order Root → Left → Right.

7. postorder(i) – Traverses the tree in the order Left → Right → Root.

8. delete(x) – Removes the specified value and rebuilds the tree using the remaining values.

9. while True – Displays the menu repeatedly until the user chooses Exit.

10. if-elif conditions – Perform insertion, searching, deletion, traversal, or exit based on the user's choice.

WEEK -10

10.AIM: Write a python program to implement an avl tree with all its operations

PROGRAM:

```
class Node:
    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None
        self.height = 1


class AVLTree:

    # Find height of a node
    def height(self, node):
        if node is None:
            return 0
        return node.height

    # Calculate balance factor
    def get_balance(self, node):
        if node is None:
            return 0
        return self.height(node.left) - self.height(node.right)

    # Right Rotation
    def right_rotate(self, y):
        x = y.left
        T2 = x.right

        x.right = y
        y.left = T2

        y.height = 1 + max(self.height(y.left),
                           self.height(y.right))
        x.height = 1 + max(self.height(x.left),
                           self.height(x.right))

        return x

    # Left Rotation
    def left_rotate(self, x):
```

```
    y = x.right
    T2 = y.left

    y.left = x
    x.right = T2

    x.height = 1 + max(self.height(x.left),
                       self.height(x.right))
    y.height = 1 + max(self.height(y.left),
                       self.height(y.right))

    return y

# Insert a node
def insert(self, root, key):
    if root is None:
        return Node(key)

    if key < root.key:
        root.left = self.insert(root.left, key)
    elif key > root.key:
        root.right = self.insert(root.right, key)
    else:
        return root

    root.height = 1 + max(self.height(root.left),
                          self.height(root.right))

    balance = self.get_balance(root)

    # LL Case
    if balance > 1 and key < root.left.key:
        return self.right_rotate(root)

    # RR Case
    if balance < -1 and key > root.right.key:
        return self.left_rotate(root)

    # LR Case
    if balance > 1 and key > root.left.key:
        root.left = self.left_rotate(root.left)
        return self.right_rotate(root)

    # RL Case
    if balance < -1 and key < root.right.key:
        root.right = self.right_rotate(root.right)
```

```
        return self.left_rotate(root)

    return root

# Find minimum value node
def min_value_node(self, root):
    current = root
    while current.left is not None:
        current = current.left
    return current

# Delete a node
def delete(self, root, key):
    if root is None:
        return root

    if key < root.key:
        root.left = self.delete(root.left, key)

    elif key > root.key:
        root.right = self.delete(root.right, key)

    else:
        if root.left is None:
            return root.right
        elif root.right is None:
            return root.left

        temp = self.min_value_node(root.right)
        root.key = temp.key
        root.right = self.delete(root.right, temp.key)

    if root is None:
        return root

    root.height = 1 + max(self.height(root.left),
                          self.height(root.right))

    balance = self.get_balance(root)

    # LL Case
    if balance > 1 and self.get_balance(root.left) >= 0:
        return self.right_rotate(root)

    # LR Case
    if balance > 1 and self.get_balance(root.left) < 0:
```

```
        root.left = self.left_rotate(root.left)
        return self.right_rotate(root)

    # RR Case
    if balance < -1 and self.get_balance(root.right) <= 0:
        return self.left_rotate(root)

    # RL Case
    if balance < -1 and self.get_balance(root.right) > 0:
        root.right = self.right_rotate(root.right)
        return self.left_rotate(root)

    return root

# Search for a key
def search(self, root, key):
    if root is None or root.key == key:
        return root

    if key < root.key:
        return self.search(root.left, key)

    return self.search(root.right, key)

# Inorder Traversal
def inorder(self, root):
    if root:
        self.inorder(root.left)
        print(root.key, end=" ")
        self.inorder(root.right)

# Preorder Traversal
def preorder(self, root):
    if root:
        print(root.key, end=" ")
        self.preorder(root.left)
        self.preorder(root.right)

# Postorder Traversal
def postorder(self, root):
    if root:
        self.postorder(root.left)
        self.postorder(root.right)
        print(root.key, end=" ")

# Find maximum value
```

```
    def maximum(self, root):
        if root is None:
            return None
        while root.right:
            root = root.right
        return root.key

    # Display height
    def display_height(self, root):
        return self.height(root)


# Main Program
tree = AVLTree()
root = None

while True:
    print("\n\n--- AVL TREE OPERATIONS ---")
    print("1. Insert")
    print("2. Delete")
    print("3. Search")
    print("4. Inorder Traversal")
    print("5. Preorder Traversal")
    print("6. Postorder Traversal")
    print("7. Find Minimum")
    print("8. Find Maximum")
    print("9. Find Height")
    print("10. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        key = int(input("Enter value to insert: "))
        root = tree.insert(root, key)
        print("Node inserted successfully.")

    elif choice == 2:
        key = int(input("Enter value to delete: "))
        if tree.search(root, key):
            root = tree.delete(root, key)
            print("Node deleted successfully.")
        else:
            print("Value not found.")

    elif choice == 3:
        key = int(input("Enter value to search: "))
```

```
        if tree.search(root, key):
            print("Value found in AVL Tree.")
        else:
            print("Value not found.")

    elif choice == 4:
        print("Inorder Traversal: ", end="")
        tree.inorder(root)

    elif choice == 5:
        print("Preorder Traversal: ", end="")
        tree.preorder(root)

    elif choice == 6:
        print("Postorder Traversal: ", end="")
        tree.postorder(root)

    elif choice == 7:
        if root:
            print("Minimum value:", tree.min_value_node(root).key)
        else:
            print("Tree is empty.")

    elif choice == 8:
        print("Maximum value:", tree.maximum(root))

    elif choice == 9:
        print("Height of AVL Tree:", tree.display_height(root))

    elif choice == 10:
        print("Exiting program...")
        break

    else:
        print("Invalid choice. Please try again.")
```

OUTPUT:



ERROR TABLE:

ERROR	RECTIFICATION
RecursionError: maximum recursion depth exceeded	Ensure recursive functions have proper base conditions.
Could not insert key	Used a suitable table size or probing technique to provide more available positions.

EXPLANATION:

1. 1.Node Creation: Creates nodes with key, left child, right child and height.
2. AVLTree Class: Defines all AVL Tree operations.
3. Height Calculation: Returns the height of a node.
4. Balance Factor: Calculates the difference between left and right subtree heights.
5. Right Rotation: Balances a left-heavy tree.
6. Left Rotation: Balances a right-heavy tree.

7. Insertion: Inserts a new node and balances the tree using rotations.
8. Deletion: Removes a node and restores tree balance.
9. Searching: Searches for a given value in the tree.
10. Inorder Traversal: Displays elements in ascending order.
11. Preorder Traversal: Visits root, left subtree and right subtree.
12. Postorder Traversal: Visits left subtree, right subtree and root.
13. Minimum and Maximum: Finds the smallest and largest elements.
14. Height Display: Displays the height of the tree.
15. Main Program: Provides a menu-driven interface to perform all operations.

WEEK-11

11.AIM: Python program to implement Hashing (Separate chaining, Linear probing, and Quadratic probing).

PROGRAM:

```
#AV.SC.U4CSE25103 POOJASREE

SIZE = 10

# 1. Separate Chaining
def separate_chaining(keys):
    table = [[] for i in range(SIZE)]

    for key in keys:
        index = key % SIZE
        table[index].append(key)

    print("\nSeparate Chaining:")
    for i in range(SIZE):
        print(i, ":", table[i])

# 2. Linear Probing
def linear_probing(keys):
    table = [None] * SIZE

    for key in keys:
        index = key % SIZE

        while table[index] is not None:
            index = (index + 1) % SIZE

        table[index] = key

    print("\nLinear Probing:")
    for i in range(SIZE):
        print(i, ":", table[i])

# 3. Quadratic Probing
def quadratic_probing(keys):
    table = [None] * SIZE

    for key in keys:
```

```
        index = key % SIZE
        i = 0

        while i < SIZE:
            new_index = (index + i * i) % SIZE

            if table[new_index] is None:
                table[new_index] = key
                break

            i += 1

        if i == SIZE:
            print("Could not insert", key)

    print("\nQuadratic Probing:")
    for i in range(SIZE):
        print(i, ":", table[i])

# Main Program
keys = list(map(int, input("Enter the keys: ").split()))

separate_chaining(keys)
linear_probing(keys)
quadratic_probing(keys)
```

OUTPUT:



ERROR TABLE:

ERROR	RECTIFICATION
NameError: name 'separate_chaining' is not defined	Define the separate_chaining() function before calling it.
Could not insert key	Used a suitable table size or probing technique to provide more available positions.

EXPLANATION:

1. Initialize the hash table size as 10.
2. Define a hash function using key % SIZE to calculate the index for each key.
3. In Separate Chaining, create an empty list at every index of the hash table.
4. Insert each key into its calculated index. If multiple keys have the same index, store them in the same list.
5. In Linear Probing, initialize the hash table with None values.
6. Calculate the hash index for each key. If the position is occupied, move to the next index using (index + 1) % SIZE until an empty position is found.
7. In Quadratic Probing, calculate the hash index for each key. If a collision occurs, find an empty position using (index + i * i) % SIZE.
8. Insert the keys into their respective positions in the hash table.
9. Display the hash tables obtained using all three collision resolution techniques.
10. Finally, take integer keys as input from the user and execute all three hashing methods.